

Live Map Flow Guide

How the Live VPR Map Is Built, Stored, and Queried

Project Documentation

April 22, 2026

Purpose

This document explains the **map flow** used by the live VPR demo. It answers four practical questions:

- How is a live reference map created?
- What exactly is saved to disk?
- How are descriptors extracted and stored?
- How are live query frames matched back to the map?

It is written as an onboarding document for a new developer. The goal is to make the control flow easy to follow without copying entire source files into the document.

1 The Short Answer

The live system stores the map in a **.npz** file containing:

- a matrix of normalized reference descriptors,
- a list of filesystem paths to the saved reference images,
- and metadata describing how the map was built.

For a **live-built map**, the system also keeps two external artifacts on disk:

- the recorded traversal video, and
- the sampled reference image files extracted from that video.

So the map is **not** just images, and it is **not** just features either. It is a combination of:

- **descriptors in the map file** for fast matching,
- **image files on disk** for visualization and debugging,
- **metadata** to make the map reusable and self-describing.

2 End-to-End Architecture

2.1 Overview

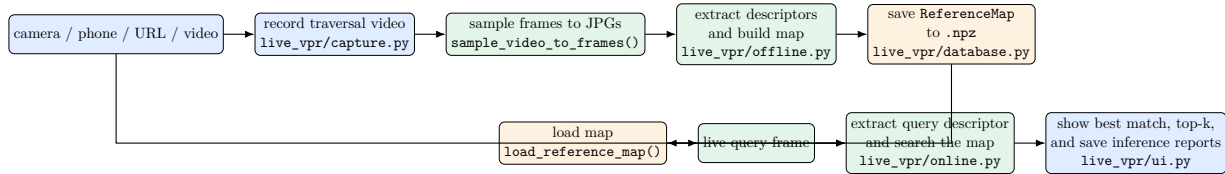


Figure 1: The live map flow: capture once, build once, then localize many query frames against the saved map.

2.2 Artifact-Level View

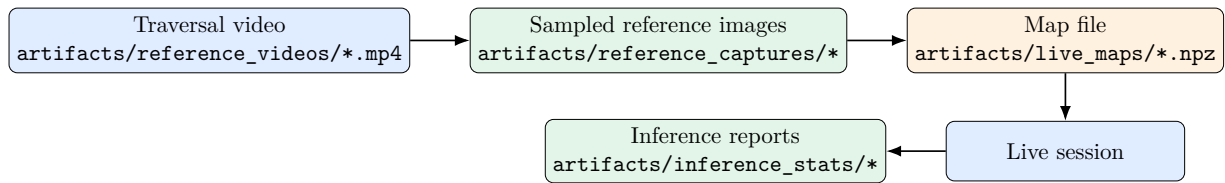


Figure 2: Persistent artifacts used by the live map system.

3 Which Files To Read

If you want to understand the flow in code, read these files in this order:

1. `live_vpr_test.py`
2. `live_vpr/capture.py`
3. `live_vpr/offline.py`
4. `live_vpr/database.py`
5. `live_vpr/extractors.py`
6. `live_vpr/online.py`
7. `live_vpr/search.py`
8. `live_vpr/ui.py`

4 Step 1: The CLI Starts the Live Map Build

The live map flow begins in `live_vpr_test.py`, inside `build_live_map()`.

Listing 1: Core structure of `build_live_map()` in `live_vpr_test.py`.

```

sampling_result = sample_video_to_frames(
    FrameSamplingConfig(

```

```

        video_path=recording_video_path,
        output_dir=args.capture_dir,
        sample_fps=args.sample_fps,
        frame_prefix=args.capture_prefix,
        max_frames=args.max_captures,
        save_scale=args.capture_save_scale,
    )
)

builder = MapBuilder(args.descriptor)
reference_map, stats = builder.build_from_paths(
    image_paths=image_paths,
    output_path=args.map_path,
    image_dir=sampling_result.output_dir,
    target_size=(args.resize_width, args.resize_height),
    metadata_extra={...},
)

```

What this means:

- The CLI either records a traversal video or reuses an existing video.
- It samples that video into a folder of still reference images.
- It then passes those saved image paths into `MapBuilder`.
- `MapBuilder` extracts descriptors and writes the final `.npz` map.

This function is the best place to start because it shows the full orchestration in one place.

5 Step 2: Traversal Video Recording

If the user is building the map from a live source, the video is recorded by `LiveReferenceRecorder` in `live_vpr/capture.py`.

Listing 2: Traversal recording loop in `live_vpr/capture.py`.

```

while True:
    ok, frame = self.source.read()
    if not ok or frame is None:
        break

    if writer is None:
        writer = cv2.VideoWriter(...)

    if recording:
        writer.write(frame)
        frame_count += 1

```

Important detail: the raw traversal frames are first stored as a normal `.mp4` video. This is useful because:

- the traversal can be reused later,

- different sampling rates can be tested without walking the route again,
- and the video acts as a reproducible record of how the map was created.

6 Step 3: Sampling the Video Into Reference Images

After recording, the video is converted into a set of sampled reference images by `sample_video_to_frames()` in `live_vpr/capture.py`.

Listing 3: Frame sampling and saved-image downsampling.

```
if should_sample:
    filename = f"{config.frame_prefix}_{len(saved_paths):04d}_{timestamp_ms:08d}ms.jpg"
    output_path = output_dir / filename
    frame_to_save = frame
    if config.save_scale < 1.0:
        frame_to_save = cv2.resize(frame, (new_width, new_height),
                                    interpolation=cv2.INTER_AREA)
    cv2.imwrite(str(output_path), frame_to_save)
    saved_paths.append(str(output_path))
```

What is happening here:

- The video is sampled by time, not by hardcoded frame index.
- Each sampled frame is saved as a real image file in `args.capture_dir`.
- The file name includes a frame counter and timestamp.
- The new `save_scale` option lets the project store smaller sampled images to reduce disk and memory pressure.

Practical note. These sampled images are not just temporary scratch files. The rest of the system keeps their paths and later uses them in the live UI for match thumbnails and exported reports.

7 Step 4: Loading Images and Extracting Descriptors

The actual descriptor-building logic lives in `live_vpr/offline.py`, inside `MapBuilder.build_from_paths()`.

Listing 4: Map building in `live_vpr/offline.py`.

```
images = [_load_rgb_image(path, target_size) for path in image_paths]

descriptors = compute_global_descriptors(self.extractor, images)
descriptors = normalize_descriptors(descriptors)

reference_map = ReferenceMap(
    descriptors=descriptors,
    image_paths=[str(path.resolve()) for path in image_paths],
    metadata=metadata,
)
saved_path = save_reference_map(reference_map, output_path)
```

This step does four things:

1. Load each sampled image from disk.
2. Convert it to RGB and resize it to the map target size.
3. Extract one global descriptor per image.
4. Normalize the descriptor matrix so cosine-style similarity becomes a dot product.

The image-loading helper is intentionally simple:

Listing 5: Image preprocessing before descriptor extraction.

```
with Image.open(image_path) as image:
    image = image.convert("RGB")
    image = image.resize(target_size, Image.Resampling.BILINEAR)
    return np.array(image)
```

This means the map descriptors are built from consistently resized RGB images, even if the saved sampled JPGs were originally larger or smaller.

8 Step 5: How the Feature Extractor Is Chosen

The project uses `live_vpr/extractors.py` as the adapter layer between the live pipeline and the actual feature extraction implementations under `feature_extraction/`.

Listing 6: Extractor factory in `live_vpr/extractors.py`.

```
def create_feature_extractor(descriptor_name: str) -> Any:
    if descriptor_name == "CosPlace":
        return CosPlaceFeatureExtractor()
    if descriptor_name == "EigenPlaces":
        return EigenPlacesFeatureExtractor()
    ...
```

Listing 7: Descriptor computation wrapper.

```
def compute_global_descriptors(extractor: Any, images: list[np.ndarray]) -> np.ndarray:
    descriptors = extractor.compute_features(images)
    if isinstance(descriptors, tuple):
        descriptors = descriptors[0]
    descriptors = np.asarray(descriptors, dtype=np.float32)
    if descriptors.ndim != 2:
        raise ValueError(...)
    return descriptors
```

Why this wrapper exists:

- different models expose slightly different outputs,
- the live pipeline wants a consistent 2D matrix shape,
- and the rest of the code should not need to know model-specific quirks.

9 Step 6: What a Saved Map Actually Contains

The in-memory map object is defined in `live_vpr/database.py`.

Listing 8: ReferenceMap schema.

```
@dataclass
class ReferenceMap:
    descriptors: np.ndarray
    image_paths: list[str]
    metadata: dict[str, Any]
```

The save function writes it as a compressed `.npz`:

Listing 9: Serialization in `live_vpr/database.py`.

```
np.savez_compressed(
    output,
    descriptors=reference_map.descriptors.astype(np.float32),
    image_paths=np.array(reference_map.image_paths),
    metadata=json.dumps(reference_map.metadata),
)
```

9.1 What Is Stored

Stored item	Meaning
<code>descriptors</code>	One normalized feature vector per reference image. This is the main data used for matching.
<code>image_paths</code>	Absolute paths to the sampled or original reference images on disk. These are used by the UI and report-export code.
<code>metadata</code>	Descriptor name, resize target, source directory, creation time, and extra live-build facts such as sampling FPS and traversal video path.

9.2 What Is Not Stored Inside the Map File

- The reference image pixels themselves are **not** embedded inside the `.npz`.
- The traversal video is **not** embedded inside the `.npz`.

That design keeps the map file smaller while still preserving enough information for matching and UI rendering.

10 Step 7: Metadata Added by Live Map Building

Live-built maps add extra metadata on top of the standard map fields. The extra values are attached in `live_vpr_test.py` when calling `MapBuilder.build_from_paths()`.

Listing 10: Live-build metadata extras.

```
metadata_extra={
    "live_build_video_path": recording_video_path,
    "live_build_sample_fps": sampling_result.sample_fps,
    "live_build_source_fps": sampling_result.source_fps,
    "live_build_video_duration_s": sampling_result.video_duration_s,
    "live_build_saved_frame_scale": args.capture_save_scale,
}
```

This is useful for reproducibility: a developer can inspect a map later and understand **how** it was created.

11 Step 8: Loading the Map During Live Inference

When the demo enters online mode, `live_vpr_test.py` loads the saved map first:

Listing 11: Map loading at runtime.

```
reference_map = load_reference_map(args.map_path)
target_size = tuple(reference_map.metadata.get("target_size", [640, 480]))
search_config = SearchConfig.from_namespace(args)

localizer = LiveLocalizer(
    reference_map=reference_map,
    descriptor_name=descriptor,
    threshold=args.threshold,
    top_k=args.top_k,
    search_config=search_config,
)
```

Two important ideas appear here:

- The live query frame is resized to the same target size used during map building.
- The descriptor type used at runtime is resolved from the map, so the system can localize with the same representation the map was built with.
- The map lookup strategy is not hardcoded; it is configured through `SearchConfig`.

12 Step 9: How Matching Works Online

The matching logic lives in `live_vpr/online.py`, inside `LiveLocalizer.localize_rgb()`.

Listing 12: Online matching in `live_vpr/online.py`.

```
descriptor = compute_global_descriptors(self.extractor, [rgb_image])
descriptor = descriptor / (np.linalg.norm(descriptor, axis=1, keepdims=True) + 1e-8)
search_result = self.search_backend.search(descriptor[0], top_k=self.top_k)
```

What this does:

1. Extract one descriptor for the current query frame.

2. Normalize it.
3. Hand the query descriptor to the configured search backend.
4. Receive the best match and top-k candidates back from that backend.

Because both map descriptors and query descriptors are normalized, every backend is still trying to solve the same geometric problem: find the reference descriptors with the largest cosine-style similarity to the query. The difference is *how* the candidate set is found.

12.1 The Search Layer

The backend abstraction lives in `live_vpr/search.py`. The localizer constructs it once and then reuses it for every frame:

Listing 13: Search backend construction.

```
self.search_config = search_config or SearchConfig()
self.search_backend = create_search_backend(reference_map.descriptors, self.search_config)
```

Current backends:

Backend	What it does	Where it shines
<code>exact</code>	Scores the query against the full descriptor matrix with a partial top-k selection.	Small or medium maps where simplicity and exactness matter more than raw speed.
<code>hnsw</code>	Uses a graph-based approximate nearest-neighbor index.	Larger maps where low query latency matters and a practical CPU ANN backend is needed.
<code>faiss_ivf_flat</code>	Searches only the most relevant FAISS clusters while still storing full vectors inside them.	Large static maps where you want a strong speed/accuracy tradeoff.
<code>faiss_ivf_pq</code>	Searches clustered descriptors that are also compressed with product quantization.	Very large maps where memory use matters in addition to search speed.

12.2 What Reranking Means

Approximate retrieval often works best as a two-stage pipeline:

1. a **fast candidate stage** that narrows the search to a shortlist,
2. then a **precise reranking stage** that computes exact scores only on that shortlist.

That second step is called **reranking**. In this project it is implemented by `_rerank_exact(...)`:

Listing 14: Exact reranking in `live_vpr/search.py`.

```
if self.config.rerank:
    indices, scores = _rerank_exact(self.descriptors, query, candidate_indices, top_k)
```


Why reranking helps:

- ANN backends are fast because they do *not* score every map descriptor.
- But the approximate ordering of candidates is not always the same as the true cosine ordering.
- Exact reranking restores the final ordering using the original descriptor matrix.
- It also gives more trustworthy final scores for thresholding and UI display.

Why reranking cannot do everything: if the fast stage never retrieves the true match, reranking cannot recover it. Reranking improves the *ordering of retrieved candidates*, not the *coverage of candidates that were never retrieved*. That is why `search_candidate_k` is still important.

12.3 A Tiny Worked Example

Suppose the map has eight reference descriptors $\{A, B, C, D, E, F, G, H\}$, and the live query really belongs near E .

Exact search. The system computes all eight scores and sorts them:

$$E : 0.92, F : 0.86, D : 0.71, B : 0.52, C : 0.49, G : 0.31, A : 0.18, H : 0.10$$

This is exact, simple, and easy to reason about, but it always touches the full map.

HNSW. The graph index might quickly retrieve candidates $\{F, E, D\}$. Those are already very good, so reranking them exactly produces:

$$E : 0.92, F : 0.86, D : 0.71$$

This is much cheaper than scanning all eight references if the real map has tens of thousands of entries instead of eight.

FAISS IVF Flat. Suppose the reference descriptors are partitioned into clusters and the query only probes the cluster containing $\{D, E, F, G\}$. Search is now restricted to that smaller region, but the vectors inside that region are still stored exactly. This is a strong option when the map is large and fairly static.

FAISS IVF PQ. Suppose the same relevant cluster is probed, but the descriptors inside it are stored in compressed form. This lowers memory usage, which matters when maps get very large, but it can make the approximate shortlist a bit noisier. Reranking is especially useful here because it restores the final ordering with exact scores.

12.4 When To Choose Each Backend

- Start with `exact` if the map is still small enough and you want the easiest baseline.
- Move to `hnsw` first when latency becomes the problem and you want a practical ANN backend on CPU.
- Use `faiss_ivf_flat` when the map is larger and you want a more scalable clustering-based search.
- Use `faiss_ivf_pq` when both speed and memory become real constraints.

12.5 Decision Rule

The localizer also produces a simple threshold-based recognition decision:

Listing 15: Recognition decision.

```
recognized = float(search_result.scores[0]) >= self.threshold
```

This means the runtime system does **retrieval + thresholding**, not offline benchmark evaluation. It returns:

- the best reference index,
- the best similarity score,
- whether that score passes the threshold,
- and the top-k ranked candidates.

13 Step 10: Why the Reference Image Paths Matter

The live UI uses the stored `image_paths` to reload the reference images later. This happens in `live_vpr/ui.py`.

Listing 16: Reference image loading in the UI.

```
def _load_reference_image(self, index: int) -> np.ndarray | None:
    ref_path = self.reference_map.image_paths[index]
    return cv2.imread(ref_path)
```

Those loaded reference images are then shown in:

- the live best-match panel,
- the live top-k panel,
- and the saved inference report images.

For example, the UI labels the best retrieved reference like this:

Listing 17: Best-match panel labels in `live_vpr/ui.py`.

```
label = "Best reference match" if result.recognized else "Best reference candidate"
cv2.putText(display, label, ...)
cv2.putText(display, f"Ref #{result.best_match_idx} score={result.best_score:.3f}", ...)
```

Practical implication. If the sampled reference images are deleted or moved after the map is built:

- matching can still run from the saved descriptors in the map file,
- but thumbnail panels and exported reports may no longer be able to display the reference images.

14 Two Variants of Map Building

The repo supports two ways to create a map.

14.1 Offline folder-based map

This path indexes an existing folder of reference images:

- entrypoint: `live_vpr_test.py` with `-mode build_map`
- build logic: `live_vpr/offline.py`
- storage format: same `ReferenceMap` and same `.npz` serializer

14.2 Live traversal-based map

This path records or reuses a traversal video first:

- entrypoint: `live_vpr_test.py` with `-mode build_live_map`
- capture logic: `live_vpr/capture.py`
- sampled-frame storage: `args.capture_dir`
- final map format: same `ReferenceMap` and same `.npz`

So the storage model is intentionally consistent: no matter how the map was created, the online localizer receives the same `ReferenceMap` abstraction.

15 A Tutorial Mental Model

If you are teaching this codebase to someone else, the simplest mental model is:

1. **Capture or collect reference views.**
2. **Turn those views into one descriptor matrix.**
3. **Save that matrix, plus paths and metadata, as a reusable map.**
4. **At runtime, turn each query frame into one descriptor.**
5. **Search the map matrix with the configured backend.**
6. **Optionally rerank the shortlist exactly.**
7. **Use the best score and top-k scores to display the result.**

16 Extension Points

These are the most useful places to modify the map flow:

If you want to change...	Start here
How traversal video is recorded	<code>live_vpr/capture.py</code> , <code>LiveReferenceRecorder</code>
How densely the traversal is sampled	<code>live_vpr/capture.py</code> , <code>sample_video_to_frames()</code>

Whether sampled images are downsampled	<code>live_vpr/capture.py</code> , <code>FrameSamplingConfig.save_scale</code> and CLI flag <code>-capture_save_scale</code>
How reference images are re-sized before descriptor extraction	<code>live_vpr/offline.py</code> , <code>_load_rgb_image()</code>
Which descriptor model is used	<code>live_vpr/extractors.py</code> and the underlying <code>feature_extraction/</code> implementations
What gets stored in the map file	<code>live_vpr/database.py</code>
How candidate retrieval is implemented	<code>live_vpr/search.py</code>
How matching decisions are packaged	<code>live_vpr/online.py</code> , <code>localize_rgb()</code>
How reference candidates are shown on screen	<code>live_vpr/ui.py</code>

17 Final Takeaway

The live map system is built around one core idea: **convert a reference traversal into a reusable descriptor database.**

The important design choices are:

- record and preserve the traversal video,
- sample that video into reference images,
- extract one descriptor per reference image,
- store descriptors, image paths, and metadata in a compact map artifact,
- then localize future frames by configurable search and optional exact reranking against that stored map.

If you understand `live_vpr_test.py`, `live_vpr/capture.py`, `live_vpr/offline.py`, `live_vpr/database.py`, `live_vpr/search.py`, and `live_vpr/online.py`, you understand the full live map flow.